Reflexión

Materia: Metodología I, Tecnicatura Universitaria en Programación
Integrante: Matias Limina
Fecha: 2026-10-15

1. ¿Qué información del Discovery resultó incorrecta, no verificable o inventada por el agente? ¿Cómo lo detectaron?

En la verificación manual de las fuentes encontré varias inconsistencias, y lugares donde el agente directamente rellenó con sus propias ideas. Por ejemplo, en el informe marcó como inexistentes los precios de Doctoralia, cuando estaban publicados. En otra página Argentina sí figuraba entre los países cubiertos, y el agente puso que no.

El caso más importante fue Bilog. El agente le había dado muy poco valor porque su página casi no tenía información. Revisando videos de la empresa descubrí que era un sistema muy completo. Además, uno de los puntos que pensábamos destacar en el MVP, la autogestión del paciente, sí existía en el mercado, así que no era un diferencial.

La conclusión que saco es que, cuando al agente le falta un dato, tiende a completarlo o a tomar la falta de evidencia como si la cosa no existiera. De ahí sale una regla que quedó en la base de conocimiento: "no evidenciado" no equivale a "no existe".

2. ¿En qué etapa usaron Ajustar, qué estaba mal y qué habría pasado si hubieran apretado Continuar?

Lo usé en tres momentos:

- **Discovery.** El MVP "imprescindible" que propuso el agente era muy, muy grande: odontograma, presupuestos, caja, de todo realmente. Lo recorté a un núcleo mucho más acotado, pensando en que es un trabajo para la universidad que estoy haciendo solo.
- **Base de conocimiento.** Había muchos supuestos que el agente ya había decidido por su cuenta, como que el sillón era siempre obligatorio o cómo se manejaban los sobreturnos. Los revisé y los ajusté. El caso más claro fue el del sillón, que pasó a depender de la prestación.
- **Change 3 (explore).** Aparecieron cosas que la base de conocimiento no cubría, como el solapamiento de turnos de un mismo paciente o qué hacer si el pedido trae un sillón cuando la prestación no lo usa. Todo eso se cerró antes del propose. Si hubiera apretado Continuar, el agente lo habría resuelto solo en el código, sin que nadie lo decidiera. Por ejemplo, un paciente podría haber quedado con dos turnos al mismo tiempo con profesionales distintos.

En el change 5 no hicieron falta muchos cambios, porque en el 3 ya se habían resuelto muchos de los posibles problemas y vacíos.

3. ¿Qué diferencia notaron entre implementar con una especificación aprobada y pedir código con un prompt suelto?

La spec aprobada le da un significado concreto a "terminado": no alcanza con que el código ande, tiene que cumplir lo que dice la spec. Además:

- El agente no puede inventar reglas. Si algo no está en la spec, se vuelve al propose.
- Los casos raros se deciden antes, no en el medio del código.
- El alcance queda controlado. Los changes no metieron funcionalidades extra "por si acaso" que correspondían a fases posteriores, sino que se centraron en cumplir lo que ya estaba explicitado en los documentos.

El costo es que hay que frenar y pensar antes de escribir código. El propose del change 5 salió con 41 escenarios, lo que es más lento al principio. Pero como todo estaba decidido de antemano, el apply salió casi de una, y la implementación fue mucho más rápida.

4. ¿Qué parte de la fundación (knowledge-base, roadmap, reglas) le aportó más valor al ciclo OPSX y cuál menos?

Lo que más valor me dio fue la base de conocimiento, puntualmente los archivos de decisiones y supuestos y de preguntas abiertas. Cada explore terminaba resolviendo preguntas y sumándolas a la base de conocimiento, así que el propose ya salía con eso adentro.

Las reglas duras también pesaron mucho. La restricción de exclusión en PostgreSQL con test de concurrencia, y no usar SQLite ni mocks, hicieron que la garantía de que no haya turnos solapados fuera real y no solo una validación en la aplicación.

Lo que menos valor tuvo fue el roadmap. Sirvió para elegir changes chicos, ver dependencias y saber qué dejar afuera en cada escenario. Pero partes como el plan multiagente y las compuertas de paralelismo casi no se usaron, porque fuimos change por change en secuencia. Tampoco llegué todavía a la parte del frontend, que es donde entran las skills pensadas para esas tareas.

Cierre

En general me gustó trabajar con este stack. Lo que no me agrada tanto es que sea tan horizontal: primero se construye todo el backend y recién al final el frontend. Preferiría un enfoque más vertical, con funcionalidades completas de punta a punta, para poder ir probándolas yo mismo después de cada implementación. Supongo que es una cuestión de gusto.

Una de las skills que trae el stack es de Matt Pocock, y me inclino más por el estilo de trabajo que propone él. Me gusta su idea de trabajar en slices verticales y de aprovechar la construcción de prototipos para planear con una idea más clara de lo que se quiere hacer. Este enfoque tampoco está mal, es otro estilo: es muy ordenado y creo que el resultado final va a ser muy satisfactorio.